

Library Evolution Policies

The rationale and process of setting a policy for the Standard Library

Document #:	P2267R0
Date:	2023-10-10
Project:	Programming Language C++
Audience:	WG21 Library Evolution Work Group
Reply-to:	Inbal Levi < sinbal2l@gmail.com > Ben Craig < ben.craig@gmail.com > Fabio Fracassi < fabio@fracassi.de >

Contents

- [1 Introduction](#)
- [2 Motivation and Scope](#)
 - [2.1 What is a policy](#)
 - [2.2 Rationale for setting policies](#)
 - [2.2.1 Pros for setting policies for the standard library](#)
 - [2.2.2 Cons for setting policies for the standard library](#)
 - [2.2.3 Summary](#)
- [3 Impact On the Standard](#)
- [4 Proposal](#)
 - [4.1 Requirements From Policy Papers and Discussions](#)
 - [4.1.1 Requirements from a policy paper](#)
 - [4.1.2 The process of setting a policy](#)
 - [4.1.3 The process of modifying a policy](#)
 - [4.1.4 The process of removing a policy](#)
 - [4.1.5 Excluding a paper from applying a specific policy](#)
 - [4.2 Wording for SD-9](#)
 - [What are Standard Library Policies](#)

- [Motivation for Standard Library Policies](#)
- [Excluding a proposal from applying a policy](#)
- [List of Standard Library Policies](#)
- [5 Acknowledgements](#)
- [6 Appendix A](#)
 - [6.1 Policy Examples \(NOT proposed in this document\)](#)
 - [6.2 Statistics of the Standard Library](#)
 - [6.2.1 Disclaimers](#)
 - [6.2.2 Examples \(Table 1\)](#)
- [7 References](#)

§ 1 Introduction

Discussion of the necessity of policies for the standard library has been going on for many years.

There have been a few papers addressing different aspects of it: [\[P0684R2\]](#), [\[P1655R0\]](#), [\[library-design-guidelines\]](#), [\[P2148R0\]](#), and others.

Note that the [\[requirements\]](#) section which appears in the beginning of the standard library section of the C++ working draft does not conflict with policies as the policies are focused on Library API.

This paper aims to survey the pros and cons of policies, and set up a process for setting and modifying a policy for the C++ standard library.

§ 2 Motivation and Scope

The standard library is a resource shared by the C++ community. As such, it contains parts which should support functionality from different domains.

At the same time, consistency is very important, as it allows users to have expectations while using different parts of the standard library.

This paper aims to define what a policy is, what's the scope in which it applies and how to build consensus for it.

§ 2.1 What is a policy

A “policy”, as discussed in this document, is any technical rule or technical guideline, which should be followed, as a general rule, by authors of proposals of **FUTURE** utilities for the C++ standard library.

A “utility” can be a container, an algorithm, an output / logging facility (`std::format`), a function, a header, or any other logical unit proposed to the standard library as a paper. A policy can be applied to existing utilities, but that will require an explicit paper.

In this document we only discuss the process and meaning, and do not propose any specific policies, but examples of such may be: Mark all constructors or only single parameter constructors as `explicit`, use hidden friends for ADL detection, etc. A list of examples of possible policies (Which are NOT proposed in this paper) is given in [Appendix A](#).

A policy has to be described clearly and provide coherent guideline for authors. The policy description can leave no room for interpretation, so that even if a utility does not follow a certain policy, we will have a clear understanding of what is the policy which is violated and how.

§ 2.2 Rationale for setting policies

We see both pros and cons to this approach, the following sections we will describe them.

§ 2.2.1 Pros for setting policies for the standard library

We believe the following should be considered as upsides:

§ 2.2.1.1 Policies create uniformity in users’ expectations from the behaviour of different parts of the standard library

The current state of the library is inconsistent. As of now, users have to be familiar with the details of a utility in order to use it.

We see a lot of value in making the standard library more coherent going FORWARD.

“Policies”, as suggested in this document should be applied to future proposals. While we encourage papers which suggest applying policies as a fix to already existing utilities, this is **out of scope for this proposal**.

Increasing consistency will have positive impact on multiple aspects, including: teachability, and consistency of code design throughout code bases.

“[Statistics of the Standard Library](#)” under [Appendix A](#) presents statistics for existing features in the standard library, to demonstrate inconsistencies.(NOTE: we do not suggest any changes for these features in this document, just demonstrate which type of inconsistencies **may** be decreased by setting Policies for **future** standard library utilities).

§ 2.2.1.2 Policies save time for both authors and the committee

With policies, authors can know what is expected of the paper before it reaches LEWG. This can save time, by avoiding repeating the same guidance to different authors. Moreover, in the current process, it is not uncommon for authors of a proposal to get

opposite and conflicting guidelines between different versions of a proposal, even without new information coming up. The reason being that the room may contain a different audience in each meeting. Having to re-iterate previously proposed directions can be a source of frustration and can be very time consuming for the authors, without adding to the quality of the proposal. We understand that in some cases going back from a change may be needed, and that this is part of the process, but we believe policies will be able to minimize this phenomenon, as authors will know, at least for some parts of the proposal, what they are expected to do.

Another topic that is time-consuming is locating and fixing bugs in the standard. More often than not, a discussion on a proposal focuses on specific topics and avoids iterating over other aspects of the proposal (either due to time constraints, or because the attendees don't have strong opinions on it). This can result in bugs, discovered later by LWG (in charge of library wording) or, worst case, after the standard is shipped and published. By providing guidance to the authors in advance, we can make sure things are less likely to "fall between the cracks". We believe this will help not only to save LEWG's time but also to save LWG's and the working draft editors' time, as well as the time of the committee as a whole.

§ 2.2.1.3 Policies need to be created from a shared knowledge base

As of now, discussions on "policy" level topics can happen at different times and in different rooms (not just in LEWG, but also in SGs), as they can be "spontaneously sparked" by a specific topic in the paper discussed. This has two main disadvantages:

1. Not all the stake-holders can be in the room, as the discussion is not always announced in advance.
2. It is not possible to be in all the rooms at once, the knowledge and guidelines tend to be sparse, and to be passed by second and third-hand.

Setting "formal" discussions in which a specific policy will be discussed helps in collecting the input of all the committee members and stake-holders, and can help with minimizing the amount of things left up for interpretation of the attendees, and misunderstandings that can result from such interpretations.

§ 2.2.1.4 Policies make the standardization process friendly for newcomers

In the current state, newcomers who would like to make a proposal to the standard library have no way of knowing "common guidelines" apart from going over all of the minutes from previous discussions (and even then, not everything is documented). This is a very bad way to preserve knowledge. Apart from being time consuming, learning from the minutes can lead to wrong outcomes, as minutes can be misinterpreted. We believe that there's a lot of value in making sure newcomers can benefit from the committee's "shared knowledge base". This will save time both for the authors, and for the committee.

§ 2.2.2 Cons for setting policies for the standard library

We believe the following should be considered as downsides:

§ 2.2.2.1 Policies may “push aside” domains which are in a minority representation in the committee

Some technical guidelines are not fit for all the users of C++. We believe that, unless we take an approach of “lowest common denominator” some parts of the standard can and should support these domains. This is true not just for policy discussions, but as a general property of the current process, as “counting votes” does. To avoid the majority vote running over the minority vote, we propose that a paper will be able to bypass a policy, by providing a rationale for it. We describe the details in section “[Excluding a paper from applying a specific policy](#)” in the proposal section, but we believe this is an important part of the process.

§ 2.2.2.2 Policies may enforce wrong technical solutions for some utilities proposed to the standard library

A major concern may be that “Policies” block specific technical solutions from being proposed into the standard library. This is a valid concern, and we address it by requiring a significant majority to set a policy, and by allowing bypassing a policy (with rationale).

§ 2.2.3 Summary

We are aware that reaching a library-wide agreement is challenging. However, we don’t suggest “a single rule fits all”. We suggest “a well-defined rule can be bypassed with care”. An example of such a policy **could** be:

- Policy A (example only, NOT proposed in this document): “Provide both `.at()` (throwing) and `operator[]` for every new container for the standard library.”

Assuming such a *hypotetical* policy:

1. This DOESN’T mean that container X, meant for embedded use, can’t be suggested to the standard if it avoids implementing `.at()`. It **can** do so, by explicitly stating that it bypasses “Policy A”, due to *some reasoning*.
2. This certainly DOESN’T mean that the rest of the library must provide support for exceptions.

We are aware that reaching a library-wide agreement (even for more specified rules, such as the one in the example above) will cost time. However, we believe that having such discussions once, instead of for every new proposal, saves time in the long term (and have other benefits, described above).

§ 3 Impact On the Standard

This proposal and the policies which will result from it are not proposed for the C++ standard (IS).

However, the policies will apply to the utilities which will be inserted into the standard.

§ 4 Proposal

We propose to set a new standing document (SD-9), which will contain all default policies for all papers reviewed by LEWG. Authors of **new papers** will have to apply these policies, unless they provide explicit rationale for avoiding it (as described in section: “[Excluding a paper from applying a specific policy](#)”). Policies will be set using the process described in section “[Requirements From Policy Papers and Discussions](#)”, and will be added into SD-9. An initial draft for SD-9 is included under section “[Wording for SD-9](#)”.

§ 4.1 Requirements From Policy Papers and Discussions

§ 4.1.1 Requirements from a policy paper

A policy paper is a paper for adding, changing or updating a policy.

Being significant to the committee process, the paper must contain the following:

1. A rationale.
2. A survey of the prior art of the topic, as appears in the standard.
3. A survey of the status quo for this topic, in the wider C++ community. Preferably, this should contain the impact on different domains and industries.
4. A clear and concise definition of the policy proposed.
5. A proposed changes to the wording of SD-9.

The wording shall include a link back to the paper making the proposal (including revision) so that it is easier to determine what was known at the time of the change for purposes of “new information”.

§ 4.1.2 The process of setting a policy

Since a policy has a significant impact on the standard library, we need to make sure all interested parties have a say.

We propose the following process for setting a policy:

1. LEWG will announce a policy topic for discussion at least one month prior to the mailing list deadline, to allow all the stake-holders to publish a paper for the upcoming mailing list, and either attend or send representatives.
2. All papers for or against a policy have to appear in the mailing list before the meeting.
3. Passing a policy paper requires a significant majority of the room.

4. The policy vote requires approval by an electronic poll.
5. The approved policy wording (as described in the paper) will then be added to SD-9.

§ 4.1.3 The process of modifying a policy

We propose the following process for modifying a policy:

1. In order to schedule a meeting for re-discussing a policy, there should be new information.
2. The rest of the process will follow the steps in section “The process of setting a policy”.

§ 4.1.4 The process of removing a policy

During a discussion on modifying a policy, a need to remove a policy may come up.

This should be done using the same process used for adding a policy.

§ 4.1.5 Excluding a paper from applying a specific policy

To address the concerns brought up in “[Cons for setting policies for the standard library](#)”, policies should be able to get bypassed. We believe that even if bypassed, there’s still value in a policy, for setting the common understanding, and saving time as described above.

Bypassing a policy should be done by explicitly stating the policy bypassed (and the specific way in which it’s being bypassed, if needed), as well as **detailed technical rationale and justifications** in the paper.

Prior art in the field discussed can also help with providing reasoning, but it’s the authors’ responsibility to explain why the policy can’t be successfully applied to the utility proposed into the standard library.

§ 4.2 Wording for SD-9

§ What are Standard Library Policies

A “policy” is any technical rule or technical guideline, which should be followed by authors of proposals of utilities for the C++ standard library.

Policies are set by Library Evolution Work Group (under JCT1/SC22/WG21), using the process described in: [[P2267R1](#)].

The following document describes the existing C++ Standard Library Policies. Please read it carefully before writing a proposal to LEWG.

As a rule, your paper should apply all the policies. The section “[Excluding a proposal from applying a policy](#)” describes the process that should be followed for any policy to be bypassed.

§ Motivation for Standard Library Policies

§ Pros for setting policies:

1. Policies create uniformity in users’ expectations from the behaviour of different parts of the standard library
2. Policies save time for both authors and the committee
3. Policies need to be created from a shared knowledge base
4. Policies make the standardization process friendly for newcomers

§ Cons for setting a policies:

1. Policies may “push aside” domains which are in a minority representation in the committee
2. Policies may enforce wrong technical solution for some utilities proposed for the standard library

§ Excluding a proposal from applying a policy

To address the concerns brought up above, a paper can avoid applying a policy, as long as it contains detailed technical rationale and justifications.

Prior art in the field discussed can also help with providing reasoning, but it’s the authors’ responsibility to explain why the policy can’t be successfully applied to the utility proposed into the standard library.

§ List of Standard Library Policies

(TODO)

§ 5 Acknowledgements

Thank you to the co-authors Ben Craig and Fabio Fracassi, and to Dvir Yitzchaki and Andrei Zissu for productive discussions.

§ 6 Appendix A

§ 6.1 Policy Examples (NOT proposed in this document)

Examples of topics which **MAY** be discussed as policies in LEWG (we DO NOT suggest any of them in this paper):

1. Mark all constructors or only single parameter constructors as `explicit`
2. Use hidden friends for ADL detection
3. Use a wide contract or a narrow contract for setting the rules of `noexcept`
4. Limit new utilities using concepts instead of type traits
5. Small headers vs. large headers
6. Member functions vs. free functions
7. Mandatory `[[nodiscard]]` on relevant functions

Note that the [\[requirements\]](#) section which appears in the beginning of the standard library section in the C++ working draft does not conflict with policies as the policies are focused on Library API.

They are also not discussed in [\[N4944\]](#) or [\[N4938\]](#).

§ 6.2 Statistics of the Standard Library

In the table below, we see the prevalence in the “[Containers](#)” section of the standard library (containers, adaptors and views), of features which **may** be considered as policy candidates.

§ 6.2.1 Disclaimers

- We DO NOT suggest here to fix these specific inconsistencies (they might as well be justified), but only to show that these inconsistencies **exist** (and **may** be decreased by setting Policies for **future** standard library utils).
- The information was collected as an example, we don’t guarantee there are no counting errors. Authors of the relevant proposals should verify the accuracy of the info when collecting these statistics (We will be happy to provide the collected info).

§ 6.2.2 Examples (Table 1)

Feature	First option	Second option	Third option	Fourth option	Fifth option
# of utils with <code>explicit</code> CTORs	No explicit CTORs (2)	Only regular explicit CTORs (12)	Both templated and regular explicit CTORs (7)	Only templated explicit CTORs (2)	-

Feature	First option	Second option	Third option	Forth option	Fifth option
# of utils with conditionally explicit CTORs	No conditionally explicit CTORs (20)	Both conditionally and regular explicit CTORs (1)	Only conditionally explicit CTORs (1)	Irrelevant (1)	-
“Swap” technique	constexpr free function calls constexpr member function (2)	template free function calls class template member function (16)	template free function calls class template member friend function (4)	class template member friend function only (1)	Irrelevant (2)

We could consider other aspects, example for such are:

- Number of utils constexpr CTORs
- Number of utils with noexcept CTORs (no noexcept CTORs / both noexcept and regular CTORs / noexcept CTORs only)
- Error handling in containers (exception throwing only / return value only / utilities support both / utilities with other error handling approaches)
- etc.

§ 7 References

- [library-design-guidelines] Titus Winters. 2018. Standard Library Guidelines.
<https://github.com/cplusplus/LEWG/blob/archive/library-design-guidelines.md>
- [N4938] Thomas Köppe. 2023. Working Draft, C++ Extensions for Library Fundamentals, Version 3.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4939.html>
- [N4944] Thomas Köppe. 2023. Working Draft, Standard for Programming Language C++.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4944.pdf>
- [P0684R2] Titus Winters. 2018. C++ Stability, Velocity, and Deployment Plans.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0684r2.pdf>
- [P1655R0] Zach Laine. 2019. LEWG Omnibus Design Policy Paper.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1655r0.pdf>
- [P2148R0] CJ Johnson and Bryce Adelstein Lelbach. 2020. Library Evolution Design Guidelines.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2148r0.pdf>

[P2267R1] Inbal Levi, Ben Craig, and Fabio Fracassi. 2023. Library Evolution Policies.

<https://wg21.link/P2267>

[requirements] Various. 2023. Library-wide requirements.

<https://eel.is/c++draft/requirements>